

Tejgaon College, Dhaka

Department of Computer Science and Engineering

Project Report

PRODUCT REQUIREMENTS DOCUMENT

Glyph

A Real-Time Framework for 3D Geometry Synthesis from Scalable Vector Graphics

A web-based framework for the real-time synthesis of three-dimensional geometry from SVG artwork.

SUBMITTED BY

Name	MD. Abdur Rahim Badsha
Registration	20502003332
Class Roll	2101013
Session	2020-21
Semester	4th Year, 8th Semester

SUPERVISED BY

Md. Moklesur Rahman
Assistant Professor, Dept. of CSE
Tejgaon College, Dhaka

Table of Contents

1. Executive Summary
2. Problem Statement
3. Goals and Objectives
4. Target Users and Use Cases
5. Scope
6. Technology Stack
7. Functional Requirements
8. Non-Functional Requirements
9. System Architecture
10. UI/UX Overview
11. Appendix: Glossary

1. Executive Summary

Glyph is a web-based, real-time framework that converts two-dimensional Scalable Vector Graphics (SVG) into three-dimensional geometry directly in the browser. Users upload an SVG file, and Glyph parses its paths, curves, and fills, then extrudes and renders the resulting shape as an interactive 3D model. The project targets designers, students, and hobbyists who want to explore 3D forms derived from flat vector artwork without needing specialized 3D modeling software.

The platform is built with Astro for a fast, responsive front end and React for interactive UI components, while Three.js (via WebGL) handles the geometry synthesis and real-time rendering pipeline. Users can retain the original SVG colors, apply custom flat colors, or use gradient and material-based effects, then export a still image of the rendered model as a PNG file.

This document defines the problem, goals, target users, functional and non-functional requirements, system architecture, UI/UX expectations, and delivery milestones for the Glyph project, and serves as the guiding reference for design, development, and evaluation.

2. Problem Statement

Vector graphics (SVG) are the standard format for logos, icons, and flat illustrations because they are lightweight, scalable, and easy to edit. However, turning a 2D vector illustration into a 3D asset today typically requires importing the file into heavyweight desktop software (e.g. Blender, Cinema 4D, or Adobe Illustrator with 3D plugins), manually tracing or extruding paths, and re-applying materials — a workflow with a steep learning curve that is inaccessible to non-specialists.

There is no lightweight, instant, browser-based tool that lets a user drag in an SVG and immediately see and manipulate a 3D version of it, with simple color and material controls, and export the result. Glyph addresses this gap by making SVG-to-3D synthesis a zero-install, real-time, in-browser experience.

3. Goals and Objectives

3.1 Project Goals

- Enable real-time conversion of any valid SVG file into extruded 3D geometry, rendered in the browser.
- Preserve design fidelity by supporting the original SVG fill colors as a default coloring mode.
- Give users creative control through custom solid colors, gradients, and basic 3D material effects (e.g. metallic, glass, matte).
- Provide an interactive viewport (orbit, zoom, pan) so users can inspect the generated model from any angle.
- Allow users to export their final 3D view as a shareable PNG image.
- Deliver a fully responsive experience that works across desktop, tablet, and mobile browsers.

3.2 Academic Objectives

- Demonstrate practical application of computational geometry (SVG path parsing, triangulation, extrusion) in a real product.
- Apply modern front-end architecture patterns (Astro islands + React components) to a graphics-intensive application.
- Showcase WebGL rendering concepts through Three.js, including lighting, materials, and camera controls.
- Produce a complete, demoable, portfolio-ready web application as a course deliverable.

4. Target Users and Use Cases

4.1 Primary Personas

Persona	Description	Primary Need
Graphic Designer	Creates logos and icon sets and wants a quick 3D preview for branding mockups.	Fast, faithful SVG-to-3D preview with color control.
Student / Hobbyist	Learning design or 3D concepts, experimenting with shapes.	An easy, no-install tool to visualize 3D forms.
Content Creator	Needs a 3D-looking image asset (thumbnail, social post) from a flat logo.	One-click styling and PNG export.

4.2 Core Use Cases

1. Upload an SVG logo and preview it as an extruded 3D object with its original colors.
2. Apply a custom gradient and a glass-like material to the model to create a stylized render.
3. Rotate and zoom the model to find a flattering angle, then export the frame as a PNG.
4. Use the tool on a tablet or phone during a class presentation to demo an idea live.

5. Scope

5.1 In Scope (v1.0)

- SVG file upload via drag-and-drop or file picker.
- Parsing of SVG paths, shapes, and fill colors.
- Real-time extrusion of 2D paths into 3D geometry (depth-based extrusion).
- Rendering via Three.js / WebGL with orbit, zoom, and pan camera controls.
- Color modes: retain original SVG colors, apply a single custom color, or apply a gradient.
- Basic material/effect presets (e.g. matte, glossy, metallic, glass).
- Adjustable extrusion depth and bevel controls.
- Lighting presets (studio, soft, dramatic) to enhance the render.

- PNG export of the current 3D viewport.
- Responsive layout for desktop, tablet, and mobile.

5.2 Out of Scope (v1.0)

- Export to 3D file formats (GLTF, OBJ, STL) — reserved for a future release.
- Animation or keyframing of the 3D model.
- Multi-user collaboration or cloud project storage.
- Support for raster images (PNG/JPG) as input — SVG only.
- Server-side rendering of the 3D scene; all rendering happens client-side.

6. Technology Stack

Layer	Technology	Purpose
Front-end Framework	Astro	Static-first site generation, fast page loads, island-based hydration for interactive components.
UI Components	React	Interactive controls: upload panel, color/material editor, viewport toolbar.
3D Rendering	Three.js (WebGL)	Scene graph, camera, lighting, materials, and real-time rendering of extruded geometry.
SVG Parsing	Three.js SVGLoader + custom path utilities	Converts SVG path data into shapes usable for extrusion.
Styling	CSS / Tailwind (responsive utility classes)	Responsive, mobile-first layout across breakpoints.
Export	Canvas.toBlob() / WebGL renderer snapshot	Captures the current render as a downloadable PNG.
State Management	React hooks / Context	Manages upload state, color/material settings, and viewport state.
Hosting	Static hosting (e.g. Vercel/Netlify)	Deploys the Astro build as a static, globally distributed site.

7. Functional Requirements

7.1 SVG Upload

ID	Requirement	Priority
FR-1.1	User can upload an SVG file via drag-and-drop onto the canvas area.	Must

ID	Requirement	Priority
FR-1.2	User can upload an SVG file via a standard file picker button.	Must
FR-1.3	System validates the file is a well-formed SVG and shows a clear error if not.	Must
FR-1.4	System displays a loading indicator while the SVG is parsed and converted.	Should
FR-1.5	User can replace the current SVG with a new upload without reloading the page.	Should

7.2 SVG-to-3D Conversion

ID	Requirement	Priority
FR-2.1	System parses SVG path and shape data into 2D geometry usable by Three.js.	Must
FR-2.2	System extrudes each parsed shape into 3D geometry with a configurable depth.	Must
FR-2.3	System applies bevel/edge smoothing controls to the extruded geometry.	Should
FR-2.4	Conversion happens in real time (target under 2 seconds for typical logo-sized SVGs).	Must
FR-2.5	System supports multi-path and multi-color SVGs, converting each path as a distinct mesh.	Must

7.3 Color and Material Controls

ID	Requirement	Priority
FR-3.1	Default mode retains and applies each path's original SVG fill color to its 3D mesh.	Must
FR-3.2	User can override all meshes with a single custom flat color via a color picker.	Must
FR-3.3	User can apply a two/three-stop gradient across the model.	Should
FR-3.4	User can select a material preset: matte, glossy, metallic, or glass.	Should
FR-3.5	User can adjust lighting presets that affect how materials appear.	Could
FR-3.6	Color/material changes apply instantly in the live preview without re-uploading.	Must

7.4 Viewport Interaction

ID	Requirement	Priority
FR-4.1	User can orbit the camera around the model via mouse drag or touch gesture.	Must
FR-4.2	User can zoom in/out via scroll wheel, pinch gesture, or on-screen controls.	Must
FR-4.3	User can pan the camera to reframe the model.	Should
FR-4.4	User can reset the camera to a default framing with one action.	Should
FR-4.5	Viewport auto-frames the model on load so it is fully visible.	Must

7.5 Export

ID	Requirement	Priority
FR-5.1	User can export the current viewport as a PNG image with one click.	Must
FR-5.2	Exported PNG reflects current color, material, lighting, and camera angle.	Must
FR-5.3	User can choose an export resolution (e.g. standard or high-res).	Could
FR-5.4	Exported file downloads automatically with a descriptive filename.	Should

8. Non-Functional Requirements

Category	Requirement
Performance	SVG-to-3D conversion should complete in under 2 seconds for files under 2 MB; the viewport should sustain 30+ FPS on mid-range hardware.
Responsiveness	Layout adapts across breakpoints (mobile, tablet, desktop) using a fluid, mobile-first design; the 3D viewport resizes without distortion.
Browser Compatibility	Supports latest versions of Chrome, Firefox, Safari, and Edge with WebGL enabled.
Usability	A first-time user can upload an SVG and see a 3D result within 3 interactions, without needing instructions.
Accessibility	Interactive controls are keyboard-navigable and labeled for screen readers where feasible; color contrast follows WCAG AA guidance for UI chrome.
Reliability	Invalid or unsupported SVG files fail gracefully with a clear, actionable error message.
Portability	Runs entirely client-side; no SVG file data is required to leave the browser.

9. System Architecture

9.1 High-Level Architecture

Glyph is a static, client-rendered web application. Astro generates the page shell and handles routing and static assets, while React 'islands' hydrate the interactive regions of the page (upload panel, control sidebar, and the Three.js viewport wrapper). All processing — SVG parsing, geometry generation, rendering, and PNG export — happens entirely in the browser; there is no backend server or database in v1.0.

9.2 Component Breakdown

- Upload Module (React) — handles drag-and-drop/file-picker input and SVG validation.
- SVG Parser (Three.js SVGLoader + custom utilities) — converts SVG markup into path/shape data and extracts fill colors.
- Geometry Engine (Three.js ExtrudeGeometry) — extrudes 2D shapes into 3D meshes with configurable depth and bevel.
- Material Manager (React + Three.js materials) — applies color mode (original/custom/gradient) and material presets to meshes.
- Viewport / Scene (Three.js + OrbitControls) — manages camera, lighting, and rendering loop.
- Export Module — reads pixel data from the WebGL canvas and triggers a PNG download.
- Responsive Shell (Astro layout) — page structure, breakpoints, and overall responsive behavior.

9.3 Data Flow

5. User uploads an SVG file in the browser (no network transfer required).
6. The SVG Parser reads the file as text and extracts path data and fill colors.
7. The Geometry Engine converts each path into an extruded 3D mesh.
8. The Material Manager applies the selected coloring/material mode to each mesh.
9. The Scene renders all meshes with the active lighting preset; OrbitControls let the user manipulate the camera.
10. On export, the Export Module captures the current canvas frame and downloads it as a PNG.

10. UI/UX Overview

10.1 Layout

The application uses a three-region responsive layout: a top navigation bar with the Glyph logo and export action, a left/side control panel for upload and styling options (collapsing into a bottom sheet or drawer on mobile), and a central 3D viewport that fills the remaining space.

10.2 Key Screens

- Empty State — prompts the user to drag/drop or select an SVG file, with a short explanation of what Glyph does.

- Loading State — shows a lightweight progress indicator while parsing and building geometry.
- Editor State — displays the 3D viewport alongside color, material, extrusion, and lighting controls.
- Export Confirmation — brief toast/notification confirming the PNG has been downloaded.

10.3 Responsive Behavior

- Desktop: side panel and viewport shown side by side.
- Tablet: side panel collapses to a toggleable drawer; viewport takes priority.
- Mobile: controls move to a bottom sheet; touch gestures (drag to orbit, pinch to zoom) replace mouse interactions.

11. Appendix: Glossary

Term	Definition
SVG	Scalable Vector Graphics — an XML-based format for two-dimensional vector images.
Extrusion	The process of projecting a 2D shape along a third axis to create a 3D solid.
WebGL	A browser API for rendering interactive 2D/3D graphics using the GPU.
Three.js	A JavaScript library that simplifies building WebGL-based 3D scenes.
Astro	A front-end web framework optimized for fast, content-focused sites with selective hydration.
Island (Astro)	An interactive component (e.g. React) hydrated independently within an otherwise static page.
Bevel	A smoothed or angled edge applied to the boundary of an extruded shape.
Material Preset	A predefined combination of shading properties (e.g. glossy, metallic) applied to a 3D mesh.